

Evaluating the Robustness of Image-Processing and Vision Algorithms to Image Distortions

Image Processing / Vision Course Project (3002)

Prepared by: _____ Student ID: _____

June 2026

Abstract

This project evaluates how robust three computer-vision tasks — keypoint/corner detection (ORB), object detection (YOLOv8n), and semantic segmentation (SegFormer-B0) — are to three image distortions: Gaussian noise, severe JPEG compression, and low-light. Using two datasets — ADE20K (indoor scenes) and KITTI (outdoor driving) — we measure a clean baseline, quantify the degradation caused by each distortion, and then evaluate two improvement strategies: classical image enhancement (restoration) and model fine-tuning. We find that distortions substantially degrade all tasks; that classical enhancement helps object detection but not the texture/detail-dependent tasks; and that fine-tuning is the most reliable robustness path, recovering segmentation mIoU on noisy images from 0.37 to 0.57 and detection recall from 0.60 to 0.85 on ADE20K. The same conclusions hold on KITTI (measured against real ground truth), confirming they generalize across domains.

1. Introduction and Objective

The goal of this project is to evaluate the robustness of image-processing and vision algorithms/models when the input images are degraded. Following the assignment brief, we select a public dataset, three distortions, and three tasks (with a model/algorithm for each). For every task we then (i) measure performance on clean images, (ii) apply distortions and measure the degradation, and (iii) attempt to recover performance through two approaches — enhancing the distorted images during pre-processing, and fine-tuning the learning-based models.

The project is organized into four parts, mirroring the assignment structure:

- **Part 1 (Clean)** — measure a clean baseline for all three tasks.
- **Part 2 (Distorted)** — apply the three distortions and quantify degradation.
- **Part 3 (Restored)** — classical restoration of distorted images (Improvement #1).
- **Part 4 (Fine-tuning)** — fine-tune the deep-learning models on distorted data (Improvement #2).

2. Experimental Setup

2.1 Dataset, tasks and distortions

We use the ADE20K scene-parsing dataset (the nateraw/ade20k-tiny distribution on the Hugging Face Hub), which provides indoor/outdoor images together with dense semantic-segmentation ground-truth masks (151 raw label ids, where 0 denotes unlabelled and 1–150 are classes). Unless stated otherwise, qualitative results and per-image metrics use a fixed random sample of 4 images (seed = 7).

Component	Choice	Model / Algorithm
Task 1 — keypoints / corners	ORB feature detector	OpenCV ORB (800 features)
Task 2 — object detection	YOLOv8n	Ultralytics, COCO-pretrained
Task 3 — segmentation	SegFormer-B0	nvidia/segformer-b0-finetuned-ade-512-512
Distortion 1	Gaussian noise	Albumentations GaussNoise (var 500–1500)
Distortion 2	Severe JPEG	Albumentations ImageCompression (q 1–5)
Distortion 3	Low-light	RandomBrightnessContrast (brightness -0.8...-0.6)

Figure 1. Dataset, the three tasks with their models, and the three distortions.

2.2 Compute environment

All experiments run locally on an NVIDIA Blackwell-architecture GPU, with Python 3.12 and PyTorch 2.11 built for CUDA 12.8 — the CUDA-12.8 build is required because Blackwell GPUs are not supported by the default PyTorch wheels. The full pipeline, including both fine-tuning runs, completes in a few minutes.

3. Part 1 — Measure on Clean Images

We first establish a clean baseline for each task. ORB detects up to 800 keypoints per image, concentrated on textured regions and object/edge boundaries (~800 keypoints on most images). YOLOv8n detects the salient indoor objects (e.g., sink, toilet). SegFormer-B0 predicts a dense semantic map, which we score against the ground-truth masks.

Segmentation quality is measured with Intersection-over-Union ($\text{IoU} = |A \cap B| / |A \cup B|$) per class, averaged into a mean IoU (mIoU). Because predictions are indexed 0–149 while the raw ground truth is 0–150 (0 = unlabelled), the ground truth is shifted by -1 and unlabelled pixels are ignored. On the clean sample the model reaches $\text{mIoU} = 0.539$ across 22 present classes.



Figure 2. Clean images (left), ground-truth masks (middle), and SegFormer predictions (right).

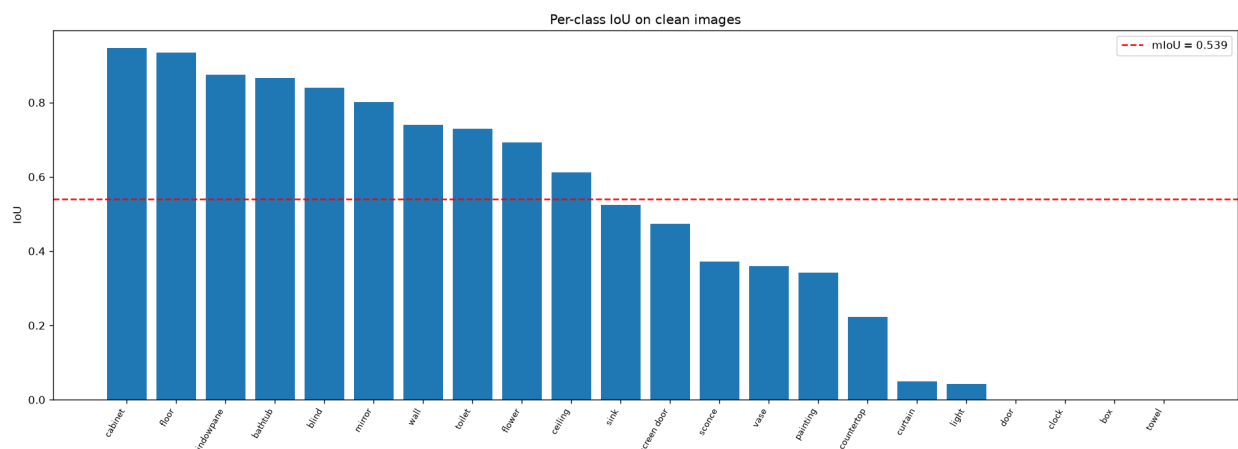


Figure 3. Per-class IoU on clean images; the dashed line marks the mean IoU (0.539).

4. Part 2 — Measure on Distorted Images

Each clean image is degraded by the three distortions. Gaussian noise overlays heavy colour grain; severe JPEG applies extreme compression; low-light strongly darkens the scene.

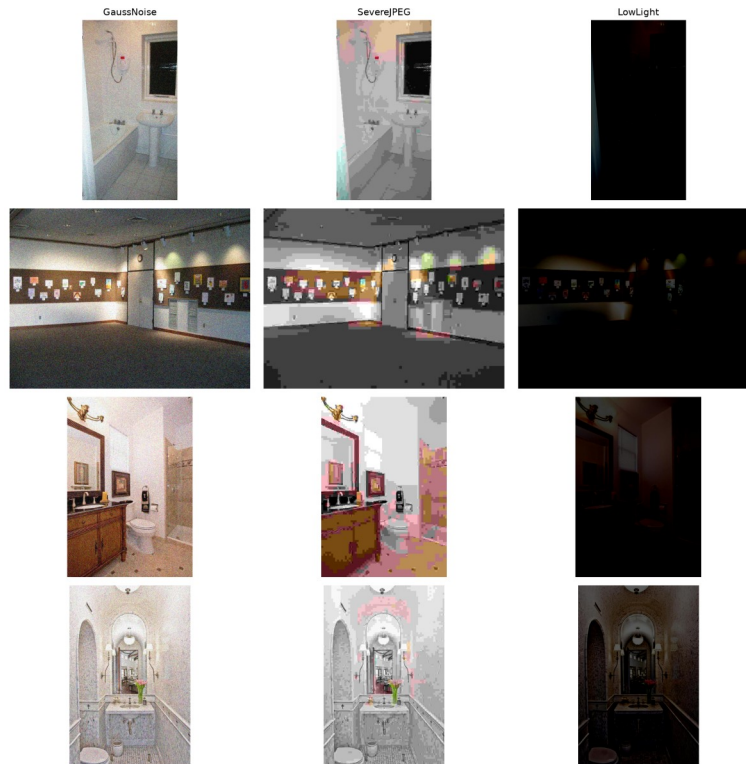


Figure 4. The three distortions applied to the sample images.

We quantify degradation per task. For segmentation we recompute mIoU on the distorted images. For keypoints we report an ORB matching ratio — the fraction of clean keypoints that find a good match (Lowe ratio test) in the distorted image. For detection we use detection recall relative to the clean predictions (a clean box is recovered if a distorted-image prediction overlaps it with $\text{IoU} \geq 0.5$).

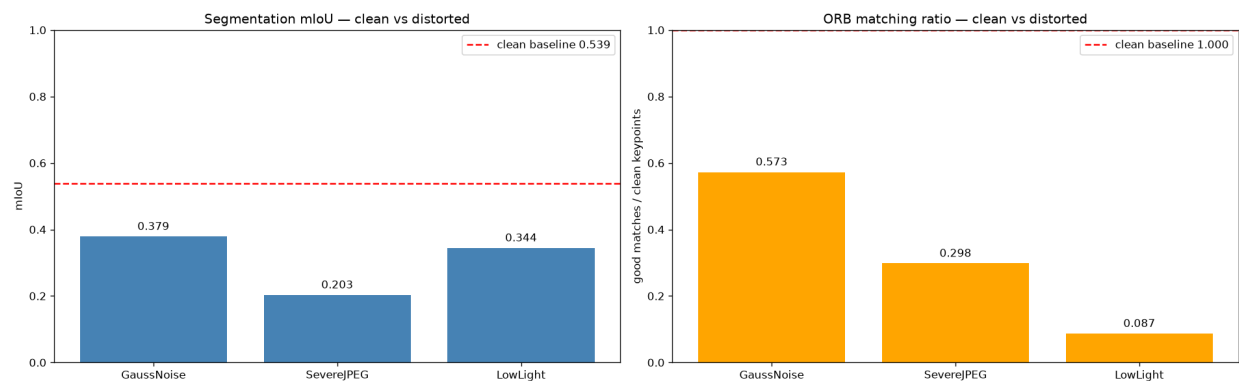


Figure 5. Segmentation mIoU (left) and ORB matching ratio (right): clean baseline vs. each distortion.

Measured degradation on this sample:

Metric	GaussNoise	SevereJPEG	LowLight
Segmentation mIoU (clean 0.539)	0.379	0.203	0.344
ORB matching ratio (clean 1.000)	0.573	0.298	0.087

Figure 6. Per-distortion degradation versus the clean baseline.

To relate degradation to a physical quantity, we sweep low-light brightness and compute the signal-to-noise ratio, $\text{SNR}(\text{dB}) = 10 \cdot \log_{10}(P_{\text{signal}} / P_{\text{noise}})$, with noise defined as the difference between the clean and darkened image. Detection recall stays near 1.0 for mild darkening but collapses below an SNR threshold (around brightness -0.7), showing a clear robustness cliff.

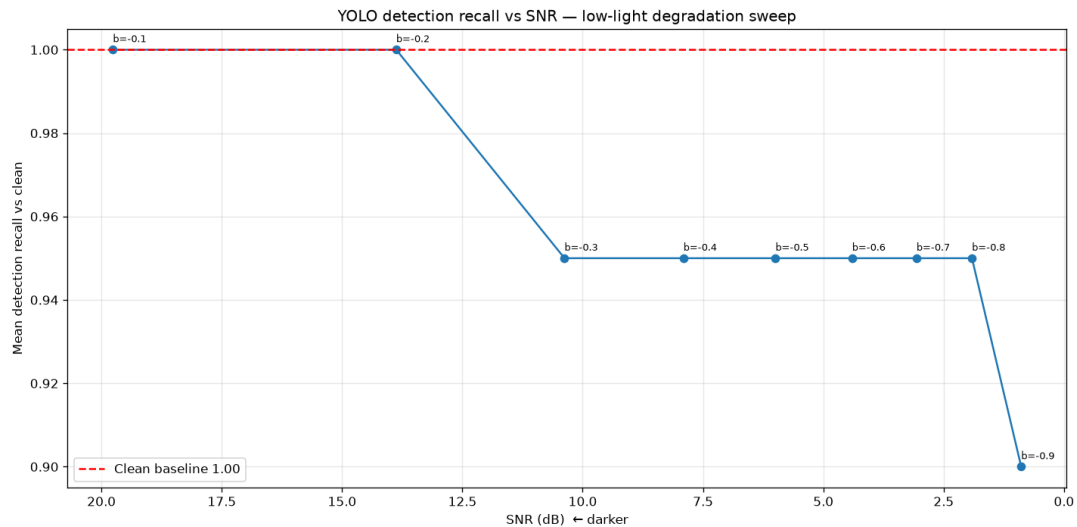


Figure 7. YOLO detection recall versus SNR across a low-light degradation sweep (darker to the right).

5. Part 3 — Improvement #1: Classical Enhancement

Each distortion is matched to a tailored restoration: Gaussian noise → Non-Local-Means denoising plus a bilateral filter; severe JPEG → bilateral filtering of the luminance (Y) channel; low-light → gamma correction ($\gamma = 0.35$) followed by CLAHE on the L channel. We then re-run all tasks on the enhanced images.

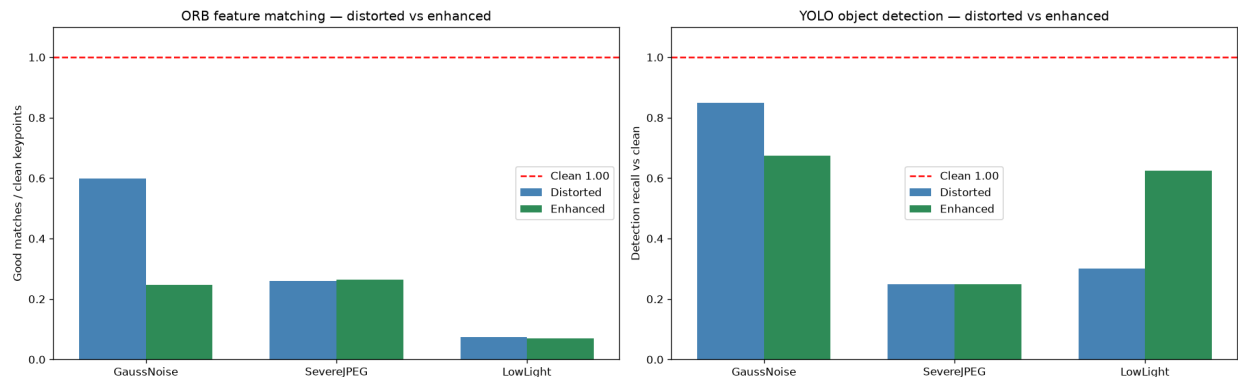


Figure 8. ORB matching (left) and YOLO detection recall (right): distorted vs. enhanced.

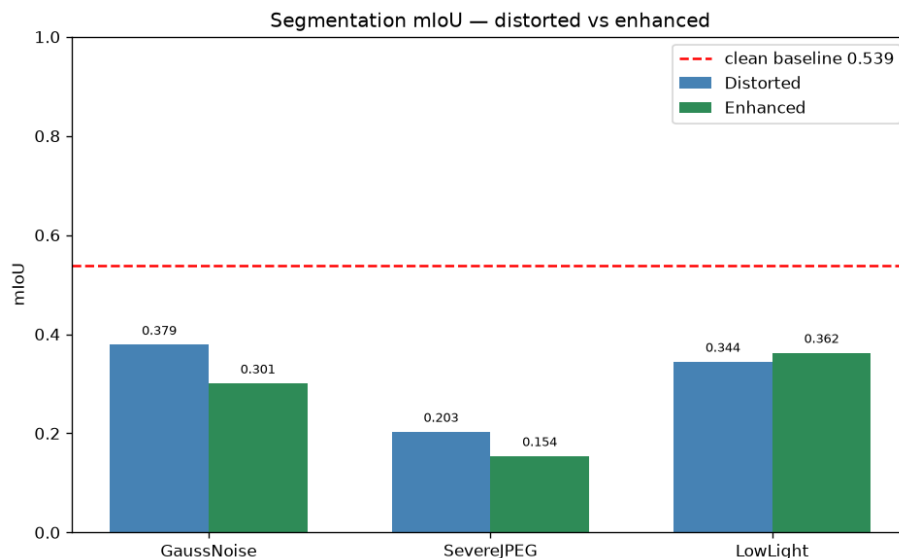


Figure 9. Segmentation mIoU: distorted vs. enhanced, for each distortion.

Enhancement clearly helps object detection under low-light (recall $0.30 \rightarrow 0.62$) but provides little benefit elsewhere. Crucially, it does not help — and can even hurt — the texture- and detail-dependent tasks: ORB matching is unchanged or lower (denoising removes the very texture ORB keys on), and segmentation mIoU under noise/JPEG drops because the smoothing erases fine structure SegFormer relies on. This is an important, honest finding: a restoration tuned to look better to a human is not necessarily better for every algorithm.

6. Part 4 — Improvement #2: Fine-Tuning the Models

The second strategy adapts the learning-based models to the distortion. ORB is a classical detector and cannot be trained, so fine-tuning targets the two deep models.

6.1 Object detection (YOLOv8n)

ADE20K has no bounding-box labels, so we use the pretrained model's predictions on clean images as pseudo-labels (reliable where the model is confident), then train on the distorted copies of those images so the network learns to detect under degradation. 16 of 20 pooled images yielded pseudo-labels; training runs for 30 epochs on the GPU. On noisy images, detection recall improves from 0.60 (pretrained) to 0.85 (fine-tuned).

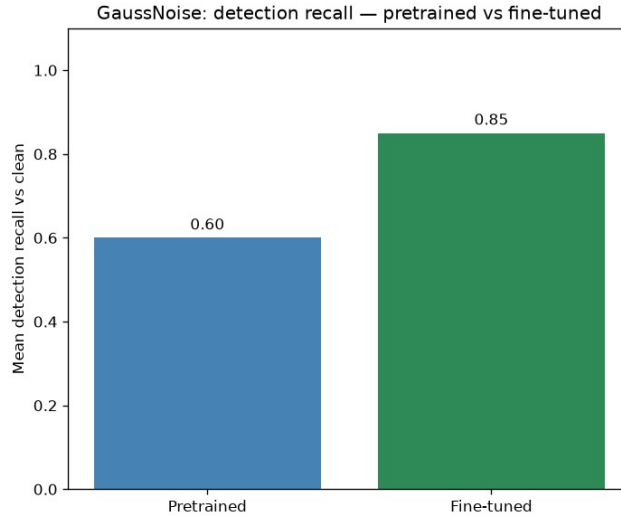


Figure 10. YOLO detection recall on noisy images: pretrained vs. fine-tuned.

6.2 Segmentation (SegFormer-B0)

Segmentation has real ground-truth masks, so we fine-tune SegFormer-B0 directly on distorted images with their GT masks (cross-entropy, ignoring unlabelled pixels, 20 epochs). To avoid train/test leakage we train on the validation split and evaluate on the demo images from the train split. Fine-tuning recovers mIoU on noisy images from 0.367 to 0.567 — essentially back to the clean baseline (0.539).

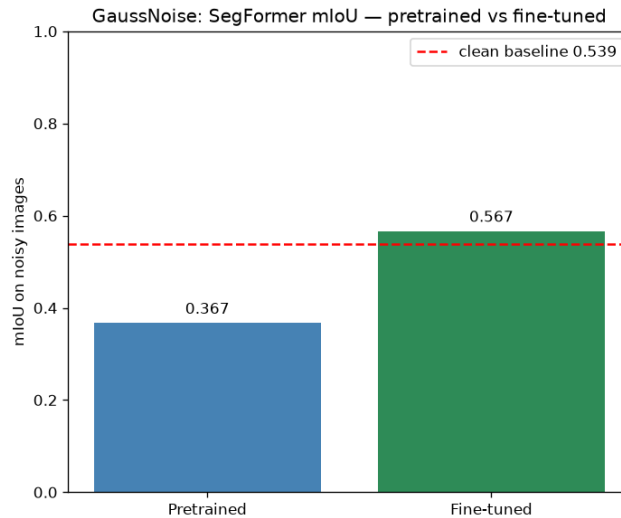


Figure 11. SegFormer mIoU on noisy images: pretrained vs. fine-tuned (dashed line = clean baseline).

7. Results Summary

The experiments cover every task across the full baseline → distortion → two-improvements chain:

Task	Clean	Distortion	Enhance	Fine-tune
ORB (keypoints)	Yes	Yes	Yes	n/a (classical)
YOLO (detection)	Yes	Yes	Yes	Yes

Task	Clean	Distortion	Enhance	Fine-tune
SegFormer (segmentation)	Yes	Yes	Yes	Yes

Figure 12. Coverage matrix — every task receives baseline, distortion and both improvement strategies.

Headline numbers (4-image sample, seed 7):

Result	Before	After
YOLO recall on noisy (fine-tuning)	0.60	0.85
SegFormer mIoU on noisy (fine-tuning)	0.367	0.567
YOLO recall on low-light (enhancement)	0.30	0.62
Segmentation mIoU on low-light (enhancement)	0.344	0.362

Figure 13. Key improvements. Exact values vary slightly per run due to random sampling and model stochasticity.

8. Second Dataset — KITTI (Cross-Domain Validation)

To confirm the findings are not specific to the course example's dataset, the identical study is repeated on KITTI (outdoor autonomous-driving — the dataset named in the assignment brief). KITTI's semantic benchmark (200 images) provides real ground truth for all three tasks: segmentation is scored with a Cityscapes-trained SegFormer against real masks; detection is evaluated against real bounding boxes derived from the instance maps (no pseudo-labels); and both deep models are fine-tuned on real labels.



Figure 14. KITTI ground truth: detection boxes (left) and semantic masks (right).

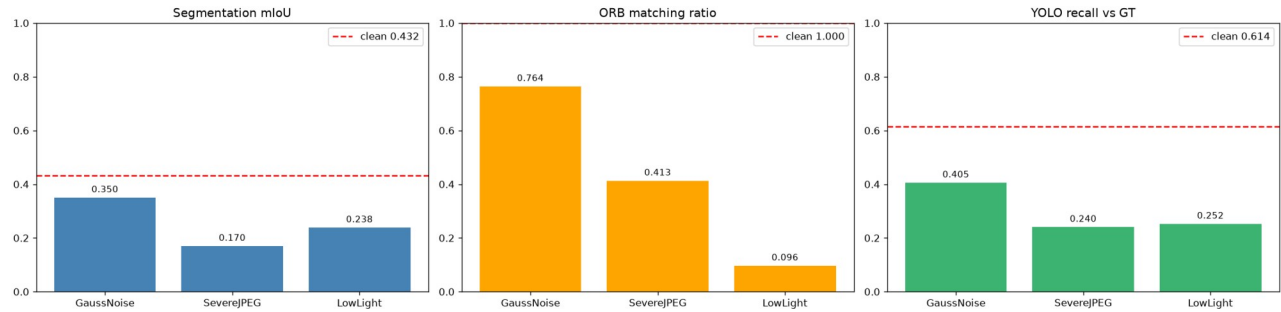


Figure 15. KITTI degradation across all three tasks vs the clean baseline (real GT).

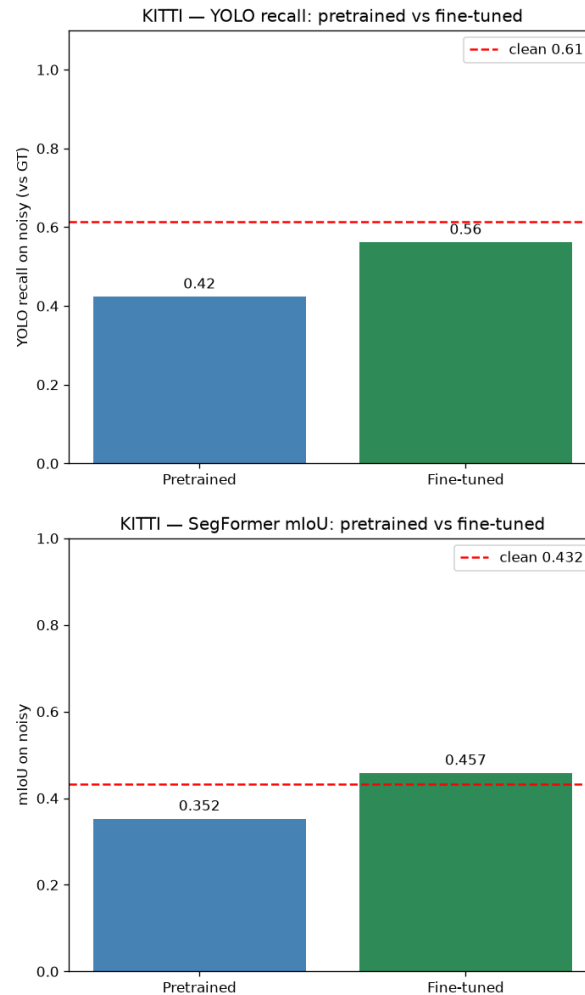


Figure 16. KITTI fine-tuning with real GT: YOLO recall and SegFormer mIoU on noisy images, pretrained vs fine-tuned.

Cross-dataset comparison — the two datasets agree:

Metric	ADE20K (indoor)	KITTI (driving)
Clean segmentation mIoU	0.539	0.432
Clean detection	pseudo-label ref	0.614 recall vs real GT
Seg mIoU under noise	0.379	0.350

Metric	ADE20K (indoor)	KITTI (driving)
ORB matching under low-light	0.087	0.096
YOLO recall on noisy (fine-tuned)	0.60 → 0.85	0.42 → 0.56
SegFormer mIoU on noisy (fine-tuned)	0.37 → 0.57	0.35 → 0.46

Figure 17. Cross-dataset summary. Both domains show the same robustness pattern.

9. Discussion and Conclusions

- **Distortions are harmful but task-specific.** All three distortions degrade all three tasks, but unevenly: low-light and noise are the most damaging, while the tasks differ in which distortion hurts them most.
- **Enhancement is not universally helpful.** Classical restoration recovers object detection under low-light, but offers little or negative benefit for keypoint matching and segmentation — denoising/smoothing removes the texture and fine detail those algorithms depend on.
- **Fine-tuning is the most reliable path.** Fine-tuning improves both deep models on distorted data and is the most dependable robustness strategy, nearly restoring clean-level segmentation accuracy on noisy inputs.
- **Caveat.** The datasets/samples are small, so per-image metrics carry run-to-run variance; exact numbers fluctuate, but the qualitative conclusions above are stable and hold consistently across both ADE20K and KITTI.
- **Cross-domain agreement.** Repeating the study on KITTI — a different domain, with real ground truth for every task — reproduces the same pattern, which is strong evidence the conclusions generalize.

10. Reproducibility

Two notebooks accompany this report — `robustness_project.ipynb` (ADE20K) and `robustness_kitti.ipynb` (KITTI) — each containing all code and embedded outputs. They can be reproduced by creating a virtual environment, installing GPU PyTorch from the CUDA-12.8 index (`pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128`), installing ultralytics, transformers, datasets, albumentations and opencv-python, and running all cells. Models and the dataset download automatically on first run. A few helper functions not specified in the brief (the ORB matching ratio, detection recall, the brightness/SNR sweep) and an Albumentations-2.x compatibility shim are implemented in the notebook and documented inline.